

Nesneye Yönelik Programlama

Hafta 7

Örnek 1

Akıllı Kütüphane Arşiv Yönetimi

Senaryo: Bir kütüphane, koleksiyonundaki kitapları dijital ortamda yönetmek istemektedir. Kitapların sadece isimleri değil; ISBN numarası ve ödünç verilme durumları da takip edilmelidir.

Sistemden beklenen gereksinimler:

- **Kitap Kaydı:** Yeni gelen kitaplar sisteme (listeye) eklenebilmelidir.
- **ISBN ile Arama:** Kullanıcı bir ISBN numarası girdiğinde, o kitabın tüm bilgileri ekrana yazdırılmalıdır.
- **Durum Güncelleme:** Bir kitap ödünç verildiğinde listedeki ilgili nesnenin "ödünçte" durumu güncellenmelidir.
- **Eski Kayıtları Temizleme:** Sayfa sayısı belirli bir değerin altında olan veya çok eski tarihli kitaplar toplu halde listeden silinebilmelidir.

Örnek 1

```
1 package hafta7;
2
3 class Kitap {
4     private String isim;
5     private String isbn;
6     private boolean oduncteMi;
7
8     public Kitap(String isim, String isbn) {
9         this.isim = isim;
10        this.isbn = isbn;
11        this.oduncteMi = false; // Başlangıçta kitap kütüphanededir
12    }
13
14    // Getter ve Setter metotları
15    public String getIsbn() { return isbn; }
16    public String getIsim() { return isim; }
17    public void setOduncteMi(boolean durum) { this.oduncteMi = durum; }
18
19    @Override
20    public String toString() {
21        return "Kitap: " + isim + " (ISBN: " + isbn + ") - Durum: " + (oduncteMi ? "Ödünçte" : "Rafta");
22    }
23 }
24
25 |
```

```

1 package hafta7;
2
3 import java.util.ArrayList;
4
5 public class KutuphaneSistemi {
6     public static void main(String[] args) {
7         ArrayList<Kitap> arshiv = new ArrayList<>();
8
9         // 1. Kitap Ekleme
10        arshiv.add(new Kitap("Java Programlama", "111-222"));
11        arshiv.add(new Kitap("Veri Yapıları", "333-444"));
12        arshiv.add(new Kitap("Algoritmalar", "555-666"));
13
14        // 2. ISBN ile Arama ve Güncelleme
15        String arananIsbn = "333-444";
16        for (Kitap k : arshiv) {
17            if (k.getIsbn().equals(arananIsbn)) {
18                System.out.println("Kitap bulundu, ödünç veriliyor...");
19                k.setOdundeMi(true); // Nesne içindeki veri güncellendi
20            }
21        }
22
23        // 3. Belirli bir kritere göre silme (Örn: İsmi "Algoritmalar" olanı sil)
24        // Listeden silme yaparken klasik for döngüsü ve index kullanımı daha güvenlidir
25        for (int i = 0; i < arshiv.size(); i++) {
26            if (arshiv.get(i).getIsim().equals("Algoritmalar")) {
27                arshiv.remove(i);
28                System.out.println("Algoritmalar kitabı arşivden çıkarıldı.");
29            }
30        }
31
32        // 4. Güncel Arşivi Listeleme
33        System.out.println("\n--- Güncel Kütüphane Listesi ---");
34        for (Kitap k : arshiv) {
35            System.out.println(k);
36        }
37    }
38 }

```

Örnek 2

Dijital Varlık (Kripto/Hisse) Portföy Yönetimi

Senaryo: Bir yatırımcı, sahip olduğu farklı dijital varlıkları (coin veya hisse) tek bir uygulamadan takip etmek istiyor. Bu senaryo, projedeki "banka hesabı ve bakiye" mantığıyla doğrudan paralellik gösterir.

Sistemden beklenen gereksinimler:

- **Varlık Ekleme:** Yatırımcı portföyüne yeni bir varlık (isim, miktar, birim fiyat) ekleyebilmelidir.
- **Portföy Değeri Hesaplama:** Tüm varlıkların (miktar * birim fiyat) toplam değeri hesaplanıp ekrana basılmalıdır.
- **Fiyat Güncelleme:** Piyasa değiştikçe listedeki varlığın birim fiyatı güncellenebilmelidir.
- **Varlık Silme (Sıfırlama):** Eğer bir varlığın miktarı 0 veya altına düşerse, o varlık portföyden (listeden) tamamen kaldırılmalıdır.

```
1 package hafta7;
2
3 import java.util.ArrayList;
4
5 class Varlik {
6     private String ad;
7     private double miktar;
8     private double birimFiyat;
9
10    public Varlik(String ad, double miktar, double birimFiyat) {
11        this.ad = ad;
12        this.miktar = miktar;
13        this.birimFiyat = birimFiyat;
14    }
15
16    public double getToplamDeger() { return miktar * birimFiyat; }
17    public String getAd() { return ad; }
18    public double getMiktar() { return miktar; }
19    public void setBirimFiyat(double yeniFiyat) { this.birimFiyat = yeniFiyat; }
20
21    @Override
22    public String toString() {
23        return ad + " -> Miktar: " + miktar + ", Değer: $" + getToplamDeger();
24    }
25 }
```

```

1 package hafta7;
2
3 import java.util.ArrayList;
4
5 public class KutuphaneSistemi {
6     public static void main(String[] args) {
7         ArrayList<Varlik> portfoy = new ArrayList<>();
8
9         // 1. Varlık Ekleme
10        portfoy.add(new Varlik("Bitcoin", 0.5, 60000));
11        portfoy.add(new Varlik("Ethereum", 2.0, 3000));
12        portfoy.add(new Varlik("Solana", 0.0, 150)); // Miktarı 0 olan bir varlık
13
14        // 2. Fiyat Güncelleme
15        for (Varlik v : portfoy) {
16            if (v.getAd().equals("Ethereum")) {
17                v.setBirimFiyat(3200.0); // Fiyat arttı
18            }
19        }
20
21        // 3. Toplam Portföy Değerini Hesaplama
22        double toplam = 0;
23        for (Varlik v : portfoy) {
24            toplam += v.getToplamDeger();
25        }
26        System.out.println("Toplam Portföy Değeri: $" + toplam);
27
28        // 4. Miktarı 0 olanları Silme (Ödevdeki hesapSil mantığı)
29        // Önemli: Silme işlemi yaparken 'i' değerini azaltmak veya sondan başlamak gerekir
30        for (int i = 0; i < portfoy.size(); i++) {
31            if (portfoy.get(i).getMiktar() <= 0) {
32                System.out.println("Uyarı: " + portfoy.get(i).getAd() + " bakiyesi 0 olduğu için listeden siliniyor.");
33                portfoy.remove(i);
34                i--; // Silme işleminden sonra index kaydığı için i'yi bir geri çekiyoruz
35            }
36        }
37
38        System.out.println("\nGüncel Portföy:");
39        for (Varlik v : portfoy) {
40            System.out.println(v);
41        }
42    }
43 }

```